

VYGO

Base de datos + Modelo de optimización de rutas multiplataforma

Documento de entrenamiento para agente de IA — Septiembre 2026

Este documento describe la arquitectura de la base de datos del sistema VYGO y el modelo matemático para la optimización de rutas de repartidores multi-plataforma.

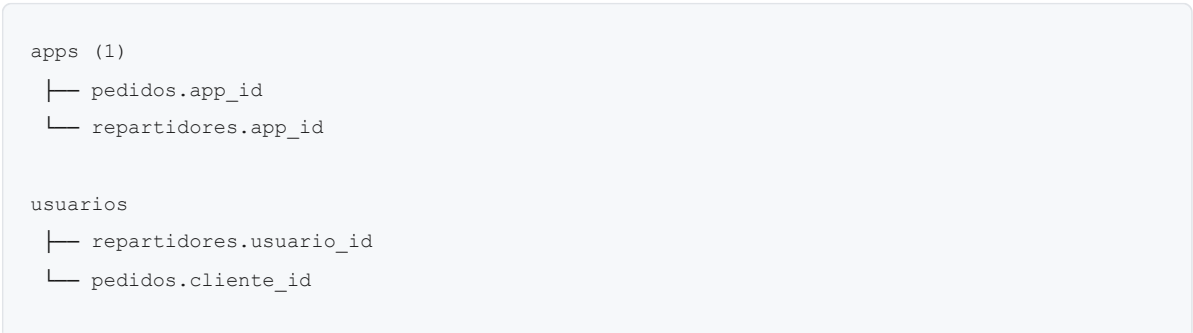
Parte I — Base de Datos VYGO

1. Visión general

Proyecto: VYGO — plataforma middleware para repartidores que trabajan simultáneamente con Uber Eats, Rappi y DiDi Food.

Motor: PostgreSQL 17 con extensión PostGIS (coordenadas geográficas en SRID 4326). **Proyecto Supabase:** `vygo` app (ref: `ihmadvmoenkxanoxrwcy` , región: `us-east-1`). **Estado:** RLS habilitado en todas las tablas. 9 tablas en esquema `public` .

2. Diagrama de relaciones



```

repartidores
├── viajes_repartidor.repartidor_id
│   └── viaje_pedidos.viaje_id → pedidos
│       ├── ofertas_pedido (pedido_id, repartidor_id, viaje_id)
│       └── difusiones_pedido.pedido_id

configuracion (tabla clave-valor independiente)
    
```

3. Tablas — Descripción completa

3.1 apps

Catálogo de plataformas de entrega conectadas al sistema.

Columna	Tipo	Nulo	Default	Descripción
id	integer	NO	nextval('apps_id_seq')	PK autoincremental
nombre	text	NO	—	Nombre único de la plataforma
activa	boolean	NO	true	Si la plataforma está operativa
creada_en	timestampz	NO	now()	Fecha de registro

Índices: apps_pkey (id), apps_nombre_key (nombre UNIQUE)

Datos actuales (3 filas): uber , didi , rappi

3.2 configuracion

Tabla de configuración global del sistema en formato clave-valor.

Columna	Tipo	Nulo	Default	Descripción
clave	text	NO	—	PK — nombre del parámetro
valor	text	NO	—	Valor del parámetro
descripcion	text	SÍ	—	Descripción legible
actualizado_en	timestampz	NO	now()	Última modificación

Datos actuales: 7 parámetros de configuración.

3.3 usuarios

Todos los actores del sistema: clientes y repartidores.

Columna	Tipo	Nulo	Default	Descripción
id	uuid	NO	gen_random_uuid()	PK
nombre	text	NO	—	Nombre completo
telefono	text	SÍ	—	Teléfono de contacto
email	text	SÍ	—	Email único
tipo	text	NO	—	'cliente' o 'repartidor'
creado_en	timestampz	NO	now()	Fecha de alta

Restricciones: tipo IN ('cliente', 'repartidor'), email UNIQUE

Índices: usuarios_pkey, usuarios_email_key

Nota importante: Esta tabla no está actualmente vinculada a auth.users de Supabase. Al conectar con producción se debe agregar columna auth_id uuid REFERENCES auth.users(id) y un trigger que cree el perfil automáticamente al registrarse.

Datos actuales: 6 usuarios (3 clientes, 3 repartidores)

3.4 repartidores

Perfil operativo del repartidor. Extiende usuarios con datos de trabajo.

Columna	Tipo	Nulo	Default	Descripción
id	uuid	NO	gen_random_uuid()	PK
usuario_id	uuid	NO	—	FK → usuarios.id
app_id	integer	SÍ	—	FK → apps.id (plataforma principal)
vehiculo	text	SÍ	—	Tipo: 'moto', 'auto', 'bici'
rating	numeric(3,2)	SÍ	5.00	Calificación promedio (0-5)
disponible	boolean	NO	true	Si está activo para recibir pedidos
creado_en	timestampz	NO	now()	Fecha de alta

Índices: repartidores_pkey, idx_repartidores_disponibles (parcial: disponible=true), idx_repartidores_usuario_id, idx_repartidores_app_id

Datos actuales: 3 repartidores (moto/Uber, auto/DiDi, bici/Rappi)

3.5 pedidos

Pedido de entrega recibido desde una plataforma externa.

Columna	Tipo	Nulo	Default	Descripción
id	uuid	NO	gen_random_uuid()	PK
app_id	integer	NO	—	FK → apps.id
id_externo	text	Sí	—	ID del pedido en la plataforma origen
cliente_id	uuid	Sí	—	FK → usuarios.id
origen	geography(Point,4326)	NO	—	Coordenadas del comercio/restaurante
origen_direccion	text	Sí	—	Dirección legible del origen
destino	geography(Point,4326)	NO	—	Coordenadas de entrega
destino_direccion	text	Sí	—	Dirección legible del destino
estado	text	NO	'buscando'	Estado actual del pedido
clima	text	NO	—	Condición climática al momento de creación
contexto	jsonb	Sí	'{}'	Metadatos adicionales (flexible)
precio	numeric(10,2)	Sí	—	Tarifa ofrecida al repartidor (MXN)
moneda	text	Sí	'MXN'	Moneda
creado_en	timestampz	NO	now()	Momento de creación
aceptado_en	timestampz	Sí	—	Momento en que fue aceptado
entregado_en	timestampz	Sí	—	Momento de entrega confirmada

Estados válidos: creado → buscando → asignado → en_camino → entregado / cancelado

Clima válido: despejado, nublado, lluvia, lluvia_fuerte, tormenta, otro

Índices: pedidos_pkey, pedidos_app_id_id_externo_key (UNIQUE), idx_pedidos_estado (parcial: activos), idx_pedidos_origen (GIST), idx_pedidos_destino (GIST), idx_pedidos_cliente_id

Datos actuales: 2 pedidos en estado `buscando`

3.6 viajes_repartidor

Viaje activo o histórico de un repartidor. Contiene la ruta actual con coordenadas PostGIS.

Columna	Tipo	Nulo	Default	Descripción
<code>id</code>	<code>uuid</code>	NO	<code>gen_random_uuid()</code>	PK
<code>repartidor_id</code>	<code>uuid</code>	NO	—	FK → <code>repartidores.id</code>
<code>origen_actual</code>	<code>geography(Point,4326)</code>	NO	—	Posición actual del repartidor
<code>destino_final</code>	<code>geography(Point,4326)</code>	NO	—	Último destino del viaje
<code>ruta_linea</code>	<code>geography(LineString,4326)</code>	SÍ	—	Geometría de la ruta completa
<code>estado</code>	<code>text</code>	NO	<code>'activo'</code>	Estado del viaje
<code>iniciado_en</code>	<code>timestampz</code>	NO	<code>now()</code>	Inicio del viaje
<code>finalizado_en</code>	<code>timestampz</code>	SÍ	—	Fin del viaje
<code>actualizado_en</code>	<code>timestampz</code>	NO	<code>now()</code>	Última actualización de posición

Estados: `activo`, `pausado`, `finalizado`

Índices: `viajes_repartidor_pkey`, `idx_viajes_activos` (parcial: `estado='activo'`), `idx_viajes_ruta` (GIST)

Datos actuales: 3 viajes activos

3.7 viaje_pedidos

Tabla pivote. Asocia pedidos a un viaje activo con orden de visita.

Columna	Tipo	Nulo	Default	Descripción
<code>id</code>	<code>uuid</code>	NO	<code>gen_random_uuid()</code>	PK
<code>viaje_id</code>	<code>uuid</code>	NO	—	FK → <code>viajes_repartidor.id</code>
<code>pedido_id</code>	<code>uuid</code>	NO	—	FK → <code>pedidos.id</code>
<code>orden</code>	<code>integer</code>	NO	<code>1</code>	Posición en la secuencia de entregas

agregado_en	timestampz	NO	now()	Cuándo se agregó al viaje
-------------	------------	----	-------	---------------------------

Restricciones: `UNIQUE(viaje_id, pedido_id)` — un pedido no puede estar dos veces en el mismo viaje

Índices: `viaje_pedidos_pkey`, `viaje_pedidos_viaje_id_pedido_id_key`, `idx_viaje_pedidos_viaje`, `idx_viaje_pedidos_pedido_id`

Datos actuales: 0 filas

3.8 ofertas_pedido

Oferta de un pedido específico a un repartidor específico. Registra si fue aceptada, rechazada o expiró.

Columna	Tipo	Nulo	Default	Descripción
id	uuid	NO	gen_random_uuid()	PK
pedido_id	uuid	NO	—	FK → pedidos.id
repartidor_id	uuid	NO	—	FK → repartidores.id
viaje_id	uuid	SÍ	—	FK → viajes_repartidor.id (si ya tiene viaje)
ronda	integer	NO	1	Ronda de difusión (aumenta si nadie acepta)
radio_metros	numeric(10,2)	NO	—	Radio de búsqueda en que fue encontrado
desvio_estimado_metros	numeric(10,2)	SÍ	—	Desvío que representa para la ruta activa
estado	text	NO	'pendiente'	Estado de la oferta
clima	text	NO	—	Clima al momento de la oferta
ofrecida_en	timestampz	NO	now()	Cuándo se ofreció
respondida_en	timestampz	SÍ	—	Cuándo el repartidor respondió
expira_en	timestampz	SÍ	—	Tiempo límite para responder

Estados: `pendiente`, `aceptada`, `rechazada`, `expirada`, `perdida`, `cancelada`

Restricción única: `(pedido_id, repartidor_id, ronda)` — no se puede ofrecer el mismo pedido dos veces en la misma ronda al mismo repartidor

Índices: `ofertas_pedido_pkey`, `idx_ofertas_pedido` (`pedido_id`), `idx_ofertas_pendientes` (parcial: pendiente por repartidor), `idx_ofertas_ronda` (`pedido + ronda`), `idx_ofertas_pedido_viaje_id`

Datos actuales: 0 filas

3.9 `difusiones_pedido`

Registro de cada ronda de búsqueda de repartidor para un pedido. El algoritmo expande el radio por rondas.

Columna	Tipo	Nulo	Default	Descripción
<code>id</code>	<code>uuid</code>	NO	<code>gen_random_uuid()</code>	PK
<code>pedido_id</code>	<code>uuid</code>	NO	—	FK → <code>pedidos.id</code>
<code>ronda</code>	<code>integer</code>	NO	—	Número de ronda (1, 2, 3...)
<code>radio_metros</code>	<code>numeric(10,2)</code>	NO	—	Radio usado en esta ronda
<code>total_ofertas</code>	<code>integer</code>	NO	—	Repartidores a los que se les ofreció
<code>clima</code>	<code>text</code>	NO	—	Clima durante la difusión
<code>iniciada_en</code>	<code>timestampz</code>	NO	<code>now()</code>	Inicio de la ronda
<code>cerrada_en</code>	<code>timestampz</code>	SÍ	—	Fin de la ronda
<code>resultado</code>	<code>text</code>	SÍ	—	Resultado final de la ronda

Resultados: `aceptada`, `sin_respuesta`, `cancelada`

Restricción: `UNIQUE(pedido_id, ronda)` — solo una difusión por ronda por pedido

Índices: `difusiones_pedido_pkey`, `difusiones_pedido_pedido_id_ronda_key`, `idx_difusiones_pedido`

Datos actuales: 0 filas

4. DDL Completo

```
-- EXTENSIÓN
-- PostGIS habilitado (geography type en uso)

-- =====
-- TABLAS
-- =====

CREATE TABLE public.apps (
    id          serial PRIMARY KEY,
    nombre      text NOT NULL UNIQUE,
```

```

    activa    boolean NOT NULL DEFAULT true,
    creada_en timestampz NOT NULL DEFAULT now()
);

CREATE TABLE public.configuracion (
    clave      text PRIMARY KEY,
    valor      text NOT NULL,
    descripcion text,
    actualizado_en timestampz NOT NULL DEFAULT now()
);

CREATE TABLE public.usuarios (
    id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre     text NOT NULL,
    telefono   text,
    email      text UNIQUE,
    tipo       text NOT NULL CHECK (tipo = ANY (ARRAY['cliente','repartidor'])),
    creado_en  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE public.repartidores (
    id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id uuid NOT NULL REFERENCES public.usuarios(id),
    app_id     integer REFERENCES public.apps(id),
    vehiculo   text,
    rating     numeric(3,2) DEFAULT 5.00,
    disponible boolean NOT NULL DEFAULT true,
    creado_en  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE public.pedidos (
    id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    app_id     integer NOT NULL REFERENCES public.apps(id),
    id_externo text,
    cliente_id uuid REFERENCES public.usuarios(id),
    origen     geography(Point,4326) NOT NULL,
    origen_direccion text,
    destino    geography(Point,4326) NOT NULL,
    destino_direccion text,
    estado     text NOT NULL DEFAULT 'buscando'
    CHECK (estado = ANY (ARRAY[
        'creado','buscando','asignado',
        'en_camino','entregado','cancelado'
    ])),
    clima      text NOT NULL
    CHECK (clima = ANY (ARRAY[
        'despejado','nublado','lluvia',
        'lluvia_fuerte','tormenta','otro'
    ])),
    contexto   jsonb DEFAULT '{}',
    precio     numeric(10,2),

```

```

moneda          text DEFAULT 'MXN',
creado_en       timestampz NOT NULL DEFAULT now(),
aceptado_en     timestampz,
entregado_en    timestampz,
UNIQUE (app_id, id_externo)
);

CREATE TABLE public.viajes_repartidor (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  repartidor_id  uuid NOT NULL REFERENCES public.repartidores(id),
  origen_actual  geography(Point,4326) NOT NULL,
  destino_final  geography(Point,4326) NOT NULL,
  ruta_linea     geography(LineString,4326),
  estado         text NOT NULL DEFAULT 'activo'
               CHECK (estado = ANY (ARRAY['activo','pausado','finalizado'])),
  iniciado_en    timestampz NOT NULL DEFAULT now(),
  finalizado_en  timestampz,
  actualizado_en timestampz NOT NULL DEFAULT now()
);

CREATE TABLE public.viaje_pedidos (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  viaje_id       uuid NOT NULL REFERENCES public.viajes_repartidor(id),
  pedido_id      uuid NOT NULL REFERENCES public.pedidos(id),
  orden          integer NOT NULL DEFAULT 1,
  agregado_en    timestampz NOT NULL DEFAULT now(),
  UNIQUE (viaje_id, pedido_id)
);

CREATE TABLE public.ofertas_pedido (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  pedido_id      uuid NOT NULL REFERENCES public.pedidos(id),
  repartidor_id  uuid NOT NULL REFERENCES public.repartidores(id),
  viaje_id       uuid REFERENCES public.viajes_repartidor(id),
  ronda          integer NOT NULL DEFAULT 1,
  radio_metros   numeric(10,2) NOT NULL,
  desvio_estimado_metros numeric(10,2),
  estado         text NOT NULL DEFAULT 'pendiente'
               CHECK (estado = ANY (ARRAY[
                 'pendiente','aceptada','rechazada',
                 'expirada','perdida','cancelada'
               ])),
  clima          text NOT NULL,
  ofrecida_en    timestampz NOT NULL DEFAULT now(),
  respondida_en  timestampz,
  expira_en      timestampz,
  UNIQUE (pedido_id, repartidor_id, ronda)
);

CREATE TABLE public.difusiones_pedido (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

pedido_id      uuid NOT NULL REFERENCES public.pedidos(id),
ronda          integer NOT NULL,
radio_metros   numeric(10,2) NOT NULL,
total_ofertas  integer NOT NULL,
clima          text NOT NULL,
iniciada_en    timestampz NOT NULL DEFAULT now(),
cerrada_en     timestampz,
resultado      text CHECK (resultado = ANY (ARRAY[
                'aceptada','sin_respuesta','cancelada'
                ])),
UNIQUE (pedido_id, ronda)
);

-- =====
-- ÍNDICES
-- =====

-- Índices condicionales (alto rendimiento en queries de producción)
CREATE INDEX idx_pedidos_estado
ON public.pedidos (estado)
WHERE estado IN ('buscando','asignado','en_camino');

CREATE INDEX idx_repartidores_disponibles
ON public.repartidores (disponible)
WHERE disponible = true;

CREATE INDEX idx_viajes_activos
ON public.viajes_repartidor (repartidor_id)
WHERE estado = 'activo';

CREATE INDEX idx_ofertas_pendientes
ON public.ofertas_pedido (repartidor_id)
WHERE estado = 'pendiente';

-- Índices geoespaciales GIST
CREATE INDEX idx_pedidos_origen ON public.pedidos USING gist (origen);
CREATE INDEX idx_pedidos_destino ON public.pedidos USING gist (destino);
CREATE INDEX idx_viajes_ruta ON public.viajes_repartidor USING gist (ruta_linea);

-- Índices de foreign keys
CREATE INDEX idx_repartidores_usuario_id ON public.repartidores (usuario_id);
CREATE INDEX idx_repartidores_app_id ON public.repartidores (app_id);
CREATE INDEX idx_pedidos_cliente_id ON public.pedidos (cliente_id);
CREATE INDEX idx_viaje_pedidos_viaje ON public.viaje_pedidos (viaje_id);
CREATE INDEX idx_viaje_pedidos_pedido_id ON public.viaje_pedidos (pedido_id);
CREATE INDEX idx_ofertas_pedido ON public.ofertas_pedido (pedido_id);
CREATE INDEX idx_ofertas_pedido_viaje_id ON public.ofertas_pedido (viaje_id);
CREATE INDEX idx_ofertas_ronda ON public.ofertas_pedido (pedido_id, ronda);
CREATE INDEX idx_difusiones_pedido ON public.difusiones_pedido (pedido_id);

-- =====

```

```
-- SEED DATA
```

```
INSERT INTO public.apps (nombre) VALUES ('uber'), ('didi'), ('rappi');
```

5. Flujo de datos — Ciclo de vida de un pedido



Parte II — Optimización de Rutas Multiplataforma

Modelo matemático, ambiente de simulación y diseño de agentes

Proyecto: capa intermedia (middleware) de agregación y ruteo entre un repartidor y múltiples plataformas de entrega (Uber Eats, Rappi, DiDi Food, paquetería).

Alcance geográfico: Zona Metropolitana de Monterrey, Nuevo León.

Naturaleza del documento: investigación y planteamiento formal. No se generan datos nuevos; se optimizan decisiones sobre datos que las plataformas ya entregan.

0. Resumen ejecutivo

El problema que estamos planteando **no es un problema de ruteo clásico**. Un VRP normal recibe un conjunto conocido de clientes y pregunta "¿en qué orden los visito para minimizar distancia?". Aquí ocurren cuatro cosas distintas al mismo tiempo:

1. **Los pedidos aparecen mientras ya vas manejando** (problema dinámico, no estático).
2. **Puedes rechazar pedidos**, y rechazar suele ser la decisión correcta (problema *selectivo* / de recolección de premios, no de cobertura obligatoria).
3. **El objetivo no es minimizar distancia sino maximizar una tasa de ganancia** (pesos por hora neta), lo que cambia la estructura matemática del óptimo.
4. **El agrupamiento (*bundling*) es la fuente real de valor**: el tiempo muerto esperando que una cocina termine de preparar es tiempo que puede pagar por sí mismo si en ese intervalo recoges otro pedido.

El documento formaliza esto en tres capas:

- **Capa 1 — Modelo determinista de referencia (MILP)**. Un "oráculo clarividente" que conoce todos los pedidos del turno por adelantado. No es implementable en producción, pero es la **cota superior** contra la cual se mide cualquier agente.
- **Capa 2 — Modelo dinámico real (SMDP con criterio de recompensa promedio)**. Es la formulación honesta del problema: decisiones dirigidas por eventos, tiempos de preparación aleatorios, tráfico y clima estocásticos. De aquí se deriva analíticamente una **regla de aceptación por umbral**.
- **Capa 3 — Sandbox y agentes**. Especificación del simulador por niveles de fidelidad (L0→L3), espacios de observación/acción con enmascaramiento de factibilidad, cinco familias de agentes con su tabla de costo/beneficio.

Recomendación técnica principal: no intentar que un solo agente de RL aprenda a la vez *qué aceptar* y *en qué orden visitar*. La arquitectura con mejor relación resultado/riesgo es **híbrida**: una política aprendida (PPO) decide **aceptar/rechazar** y un solver exacto decide **la secuencia** del conjunto ya aceptado.

1. Definición del problema

1.1 El rol del sistema

El sistema es un intermediario informacional. Recibe de cada plataforma conectada la oferta de un trabajo y devuelve al repartidor una recomendación de acción. Formalmente es un **agente de decisión secuencial bajo incertidumbre** que actúa en nombre de un solo trabajador.

Consecuencias de modelado clave:

- **Un solo vehículo**. No es un VRP multi-vehículo. Es un problema de **ruta única selectiva** (*single-vehicle profitable pickup and delivery problem with time windows*).
- **El repartidor tiene veto**. La política produce recomendaciones, no comandos. El agente debe emitir no sólo una acción sino una **justificación cuantitativa** (ganancia marginal esperada, riesgo de retraso, minutos añadidos).

1.2 Contrato de datos con las plataformas externas

Campo	Símbolo	Origen	Si no viene
Plataforma de origen	$p(i)$	dado	—
Nodo de recolección	o_i	dado (lat/lon)	—
Zona/nodo de entrega	d_i	dado	centroide de zona
Tarifa ofrecida	f_i	dado	—
Distancia estimada	δ_i	dado	calcular sobre red vial
Tiempo de preparación	r_i	a veces solo promedio	LogNormal
Fecha límite prometida	l_i	a veces implícita	derivar de ETA
Tipo de producto	$\text{tipo}(i)$	dado o inferible	caliente/frío/no perecedero
Propina	—	desconocida ex ante	tratar como ruido

Nota metodológica. La propina es desconocida al momento de decidir. Se modela como variable aleatoria de media condicionada a la zona de entrega y se optimiza el valor esperado, nunca el realizado.

1.3 El fenómeno a capturar: la ventana de holgura

Cuando el repartidor acepta el pedido i y viaja hacia o_i , llega en el instante $T_{\{o_i\}}$ pero no puede salir antes de que el producto esté listo, en r_i . La **holgura de recolección** es:

$$\sigma_i = \max\{0, r_i - T_{\{o_i\}}\}$$

Ese σ_i es tiempo pagado por nadie. El valor del *bundling* es exactamente la suma de esas holguras convertidas en ingreso.

El límite a la acumulación es doble y ambos son **duros**:

- **Frescura:** un pedido caliente recogido en s_i tolera a lo más θ_i minutos en tránsito.
- **Fecha límite:** rebasar ℓ_i degrada la calificación del repartidor.

2. Trabajo relacionado

El problema no es nuevo en su núcleo; lo nuevo es el punto de vista. **Toda la literatura de *meal delivery* está escrita desde la plataforma.** Nosotros escribimos desde **el repartidor**, que selecciona entre ofertas de varias plataformas competidoras para maximizar su propia tasa de ganancia.

Problema canónico

Qué aporta

Qué le falta

PDPTW	precedencia recolección→entrega, ventanas	obliga a servir a todos; no hay selección
TOPTW / Profitable Tour	selección de clientes con premio	no tiene pares recolección-entrega ni perecibilidad
MDRP	tiempos de preparación, agrupamiento	perspectiva de plataforma, multi-vehículo
SDVRP	llegada dinámica, formulación MDP	objetivo de costo, no de tasa de ganancia
SMDP recompensa promedio	criterio \$/hora, regla de umbral	no trae la estructura de ruteo

Nuestro problema = PDPTW + selección TOPTW + perecibilidad MDRP + dinamismo SDVRP + criterio de tasa promedio, con **un vehículo** y **múltiples fuentes de oferta no coordinadas**.

3. Capa 1 — Modelo determinista de referencia (MILP)

Propósito: **cota superior y generador de etiquetas**. Un modelo que, conociendo el turno completo *a posteriori*, calcula la ganancia máxima alcanzable.

3.1 Red física

Sea $G=(V, E)$ la red vial dirigida con V intersecciones y E segmentos. El **tiempo de viaje** es:

$$\tau_e(t, w) = (\lambda_e / v_e^0) \cdot (1 + \alpha \cdot (q_e(t) / c_e)^\beta) \cdot \gamma_w + \pi_e(t)$$

con $\alpha \approx 0.15$, $\beta \approx 4$ (función Bureau of Public Roads), $\gamma_w \geq 1$ según clima.

Consistencia FIFO: salir más tarde nunca hace que llegues antes.

3.2 Parámetros principales

Símbolo	Significado
a_i	instante en que la oferta i aparece
f_i	tarifa ofrecida por la plataforma
r_i	instante en que el producto está listo
ℓ_i	fecha límite prometida de entrega
θ_i	tolerancia máxima en tránsito (frescura)
Q	capacidad del vehículo
c_K	costo por km (combustible + mantenimiento)

ψ_i	penalización por unidad de tiempo de retraso
----------	--

3.3 Restricciones clave

(C6) **Espera por preparación** — genera la holgura σ_i :

$$S_{\{o_i\}} \geq \max\{T_{\{o_i\}}, r_i\}$$

(C7) **Precedencia recolección→entrega**:

$$T_{\{d_i\}} \geq S_{\{o_i\}} + \mu_i^P + \tau(o_i, d_i, \cdot)$$

(C9) **Frescura — restricción central**:

$$T_{\{d_i\}} - (S_{\{o_i\}} + \mu_i^P) \leq \theta_i + M \cdot (1 - y_i) \quad \forall i \in \mathcal{O}^{cal}$$

Esta desigualdad **impide el agrupamiento ilimitado** y es la formalización exacta de la prioridad: si recoges comida primero, no puedes acumular n paquetes que tomen más de lo que la comida tarda en enfriarse.

3.4 Función objetivo

$$\begin{aligned} \max \quad & \sum (f_i + g_i) \cdot y_i && (\text{ingreso}) \\ & - \sum c_k \cdot \delta(m, n, \cdot) \cdot x_{\{mn\}} && (\text{costo distancia}) \\ & - c_\tau \cdot (T_{\{0'\}} - T_0) && (\text{costo tiempo}) \\ & - \sum \psi_i \cdot z_i && (\text{retrasos}) \\ & - \sum \Psi_{\{p(i)\}} \cdot (1 - y_i) && (\text{reputación}) \end{aligned}$$

4. Capa 2 — Modelo dinámico: SMDP de recompensa promedio

4.1 ¿Por qué un SMDP?

Las decisiones no ocurren en pasos uniformes, sino **cuando pasa algo**: llega una oferta, llegas a un nodo, el producto queda listo, una oferta expira. El marco correcto es un **Proceso de Decisión Semi-Markoviano** dirigido por eventos.

4.2 Estado

$$S_k = (t_k, v_k, u_k, A_k, F_k, W_k, \theta_k, \rho_k)$$

- v_k : posición; u_k : carga a bordo
- A_k : **plan activo** = lista ordenada de paradas comprometidas

- F_k : ofertas visibles no decididas (con tiempo de expiración)
- ρ_k : tasa de ganancia realizada hasta t_k

4.3 Acción jerárquica

- **Nivel alto (aprendido):** para cada oferta pendiente, aceptar, rechazar o diferir.
- **Nivel bajo (calculado exactamente):** dado el conjunto comprometido, la secuencia óptima de visitas.

Con $|A_k| \leq 6$ pedidos (≤ 12 paradas), se enumera o resuelve con **Held–Karp** en microsegundos. No hay razón para aprender esto.

4.4 Máscara de factibilidad

Antes de exponer acciones a la política se calcula el conjunto de acciones que admiten **al menos una** secuencia factible respecto de las restricciones de frescura, capacidad y compatibilidad.

Enmascarar vs penalizar: enmascarar (no penalizar) es la diferencia entre un agente que aprende y uno que no. El gradiente de la acción enmascarada es exactamente cero, sin sesgo.

4.5 Criterio de optimalidad: tasa de ganancia

El criterio de **recompensa promedio**:

$$\rho^* = \lim_{K \rightarrow \infty} E[\sum r(S_k, A_k)] / E[\sum \Delta t_k]$$

4.6 Regla de aceptación por umbral

De la ecuación de Bellman se obtiene: **aceptar la oferta j si y solo si:**

$$[\Delta f_j - c_k \cdot \Delta \delta_j - \Delta \Psi_j] / \Delta t_j > \rho^*$$

donde Δt_j y $\Delta \delta_j$ son el tiempo y distancia incrementales que añade el pedido j al plan actual.

Tres ventajas de este resultado:

1. **Es un algoritmo listo para usar** sin entrenar nada. Es el baseline más difícil de superar.
2. **Es interpretable:** la app puede decir: *"este pedido te paga a \$128/h; tu promedio hoy es \$141/h → rechazar"*.
3. **Explica el bundling:** cuando j cabe en la holgura σ_i , el denominador $\Delta t_j \approx 0$ y la fracción explota.

Qué no captura (donde el RL debe ganar):

- **Valor de posición:** aceptar un pedido que te deja en zona de alta densidad vale más que su tasa marginal.
- **Valor de opción/espera:** a veces conviene rechazar y esperar una oferta mejor.
- **Riesgo de cascada:** agrupamientos con Δt_j pequeño en media pero alta varianza.

5. Capa 3 — Sandbox de aprendizaje

5.1 Niveles de fidelidad

Nivel	Geometría	Tiempo de viaje	Propósito
L0	rejilla 20×20, Manhattan	velocidad constante	depurar código
L1	rejilla + zonas densidad	multiplicador zona/hora	validar bundling
L2	red vial real MTY (OSMnx)	matriz por franja horaria	demo presentable
L3	red vial + microsimulación	SUMO	validación final

L0 y L1 son donde se entrena. L2 es donde se evalúa.

5.2 Red vial

```
import osmnx as ox
G = ox.graph_from_place([
    "Monterrey, Nuevo León, Mexico",
    "San Pedro Garza García, Nuevo León, Mexico",
    "San Nicolás de los Garza, Nuevo León, Mexico",
    "Guadalupe, Nuevo León, Mexico",
    "Apodaca, Nuevo León, Mexico",
    "General Escobedo, Nuevo León, Mexico",
    "Santa Catarina, Nuevo León, Mexico",
], network_type="drive", simplify=True)
G = ox.add_edge_speeds(G)
G = ox.add_edge_travel_times(G)
```

Decisión crítica de rendimiento: precalcular la **matriz de tiempos y distancias** M×M para M≈1500–3000 nodos de interés, por franja horaria. Con M=2000 y 8 franjas ≈ 128 MB en RAM. Convierte cada consulta en un acceso a arreglo.

5.3 Capa climática

Cadena de Markov (paso 15 min): {seco, lluvia_ligera, tormenta}

```
Multiplicadores  $\gamma_w$ : 1.0 / 1.15 / 1.45
Incidentes  $\lambda(w)$ :  $\lambda_0 \cdot \{1, 2, 5\}$ 
Demanda  $\Lambda(w)$ :  $\Lambda_0 \cdot \{1, 1.3, 1.6\}$ 
```

La lluvia **aumenta la demanda a la vez que aumenta el costo** — el escenario donde el agente puede diferenciarse de una heurística fija.

5.4 Espacio de observación

a) **Estado propio (≈14 dims):** posición, velocidad, carga, tiempo transcurrido, tasa actual, clima.

b) Plan activo (hasta 6 pedidos × 10 dims): desvíos relativos, tarifa, holguras de frescura y fecha límite.

c) Ofertas pendientes (hasta 8 × 12 dims): desvíos, tarifa, Δt_j y $\Delta \delta_j$ precalculados, segundos hasta expirar.

Δt_j y $\Delta \delta_j$ como features son la decisión con mayor retorno de ingeniería. Le dan a la red el numerador y denominador de la regla de umbral ya calculados. Sin ellas la red tiene que aprender geometría de rutas desde coordenadas crudas — órdenes de magnitud más costoso.

5.5 Antipatrones de recompensa

Antipatrón	Síntoma	Mitigación
Bonus por aceptar	acepta todo, entrega tarde	quitarlo
Penalización de retraso baja	ignora fechas límite	subir ψ
Penalización de retraso alta	rechaza casi todo	bajar ψ
Sin costo de tiempo	acepta tasa horaria pésima	término $-\rho^* \Delta t$
Bonus por distancia recorrida	vagabundea	nunca premiar movimiento

6. Familias de agentes

6.1 Baselines obligatorios

ID	Política	Qué mide
B0	Aleatoria factible	piso absoluto
B1	Aceptar todo lo factible (FIFO)	comportamiento del repartidor novato
B2	Regla de umbral (§4.6) con ρ^* móvil	el baseline serio
B3	Reoptimización en horizonte rodante (MILP/CP-SAT)	límite sin anticipación
B4	Oráculo clarividente (información completa)	cota superior

La cadena $B1 < B2 < B3 < B4$ es la escala de medición. La diferencia $B4 - B3$ es el valor de la anticipación y el techo máximo de lo que el RL puede aportar.

6.2 Arquitectura híbrida recomendada (Familia D)

Ofertas

|

▼

```

Filtro de factibilidad (exacto)
| prueba inserción vs frescura/capacidad
▼
Evaluador de inserción (exacto, Held-Karp)
| calcula  $\Delta t_j$ ,  $\Delta \delta_j$  óptimos
▼
Política aprendida (PPO + atención cruzada)
| decide: aceptar / rechazar / reposicionar
| aprende  $h^*(S)$ : valor posicional, de espera, riesgo cascada
▼
Secuenciador exacto → plan ejecutable
    
```

Por qué esta arquitectura:

- La **secuenciación** es combinatoria pero pequeña y determinista. Los métodos exactos la dominan.
- La **aceptación** es un problema de decisión bajo incertidumbre sobre el futuro. **Aquí y solo aquí el RL tiene ventaja estructural.**

6.3 Tabla comparativa

Agente	Esfuerzo	Techo desempeño	Riesgo	Interpretable
B2 umbral	2h	medio-alto	muy bajo	sí, total
B3 MILP rodante	5h	alto	bajo	sí
DQN enmascarado	6h	alto	medio	sí (Q = pesos)
PPO + atención	8h	alto	medio	parcial
Híbrido (D)	10h	muy alto	bajo	sí

7. Optimización del entrenamiento

Ordenado por retorno sobre esfuerzo:

1. **Hacer el entorno rápido antes de tocar el algoritmo.** Objetivo: ≥ 5.000 pasos/segundo con 16 entornos en paralelo. Las matrices precalculadas son el factor 100×.
2. **Curriculum de aprendizaje:**

Etap	Configuración	Criterio de avance
1	1 oferta, sin preparación, sin clima	tasa $\geq 90\%$ de B2
2	ofertas concurrentes, R_i aleatorio	aparece bundling
3	+ frescura y fechas límite	puntualidad $\geq 85\%$

4	+ tráfico y clima	tasa > B2
5	domain randomization completa	generaliza a test

- Arranque por imitación.** Generar ~50.000 transiciones con B3, pre-entrenar por cross-entropy, luego continuar con PPO. Reduce 3–10× las muestras necesarias y evita el colapso inicial.
- Enmascaramiento en lugar de penalización.**
- Normalización.** `VecNormalize` para observaciones y recompensas.
- Entrenar con ≥3 semillas** y reportar mediana con IQR.

8. Métricas de evaluación

Métrica primaria: $\rho = (\text{ingreso} - \text{costos}) / \text{horas_conectadas}$ (pesos/hora)

Métricas secundarias:

Métrica	Descripción
Gap vs oráculo	$1 - \rho^{\pi} / \rho^{\text{MILP}}$
Pedidos/hora	Throughput
Tasa de puntualidad	% entregados antes de ℓ_i
Violaciones de frescura	Debe ser 0 (prueba de corrección)
Factor de agrupamiento	Pedidos promedio simultáneamente a bordo
Km por pedido / Km vacíos	Eficiencia de la ruta
Utilización	% tiempo en tránsito con carga

9. El resultado de una línea

Aceptar la oferta j si y solo si:

[tasa marginal calculable exactamente] + [valor posicional y de opción aprendido] > [tu tasa de

$$(\Delta f_j - c_x \cdot \Delta \delta_j) / \Delta t_j + \Delta E[h^*(S')] / \Delta t_j > \rho^*$$

El **primer término** es lo que un repartidor experimentado ya calcula mentalmente.

El **segundo término** es lo que no puede calcular, y es lo que el sistema aporta.

El **tercer término** es lo que el sistema debe mostrarle para que confíe en la recomendación.

10. Referencias

1. Reyes et al. (2018). *The Meal Delivery Routing Problem*. Optimization Online.
2. Yildiz & Savelsbergh. *Provably High-Quality Solutions for the Meal Delivery Routing Problem*. Transportation Science.
3. Ulmer et al. (2021). *The Restaurant Meal Delivery Problem: Dynamic Pickup and Delivery with Deadlines and Random Ready Times*. Transportation Science 55(1).
4. Hildebrandt, Thomas & Ulmer (2023). *Opportunities for Reinforcement Learning in Stochastic Dynamic Vehicle Routing*. Computers & Operations Research.
5. Li et al. (2021). *Heterogeneous Attentions for Solving Pickup and Delivery Problem via Deep Reinforcement Learning*. arXiv:2110.02634.
6. Huang & Ontañón (2020). *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. arXiv:2006.14171.
7. Popan, C. (2024). *The Fragile 'Art' of Multi-Apping: Resilience and Snapping in the Gig Economy*. Environment and Planning A.
8. Kool et al. (2019). *Attention, Learn to Solve Routing Problems!* ICLR.
9. Kwon et al. (2020). *POMO: Policy Optimization with Multiple Optima for Reinforcement Learning*. NeurIPS.
10. Ng, Harada & Russell (1999). *Policy Invariance Under Reward Transformations*. ICML.
11. Puterman, M.L. (1994). *Markov Decision Processes*. Wiley.
12. Held & Karp (1962). *A Dynamic Programming Approach to Sequencing Problems*.
13. Boeing, G. (2017). *OSMnx*. Computers, Environment and Urban Systems.

Apéndice — Glosario

Símbolo	Significado
$G=(V,E)$	red vial dirigida
$\tau(u,v,t,w)$	tiempo de viaje $u \rightarrow v$ saliendo en t bajo clima w
i, O	pedido, conjunto de pedidos ofrecidos
o_i, d_i	nodos de recolección y entrega del pedido i
$a_i, r_i, \ell_i, \theta_i$	aparición, listo, fecha límite, tolerancia de frescura
f_i, g_i	tarifa, propina
σ_i	holgura de recolección (espera por preparación)
$S_k, A_k, M(S_k)$	estado, acción, acciones factibles enmascarado
A_k, F_k	plan activo, ofertas pendientes
$\rho, \rho^*, \hat{\rho}$	tasa de ganancia: de política, óptima, estimada
$h^*(S)$	función de sesgo del SMDP de recompensa promedio

c_K, c_T	costo por km, costo por minuto
$\varepsilon(\bar{v})$	rendimiento del vehículo en función de la velocidad
γ_w	multiplicador climático del tiempo de viaje
Λ_p	intensidad de llegada de ofertas de la plataforma p
K_A, K_F	cotas del plan activo y ofertas visibles

VYGO v0.1 — Documento generado el 12 de septiembre de 2026